

SportPulse

TME8 — Architecture Client et Déploiement

PC3R — M1 Informatique STL
Sorbonne Université

Feriel Benatsi
Dalila Zeghmiche

Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Architecture générale	2
3	Choix des technologies client	2
4	Plan du site et navigation	3
5	Gestion des états React	3
5.1	Chargement d'une page	4
6	Authentification JWT	4
6.1	Connexion	4
6.2	Protection des routes	4
7	Gestion des événements utilisateur	4
7.1	Publication d'un pronostic	4
7.2	Publication d'un commentaire	5
7.3	Suppression de compte	5
8	Liste des appels AJAX	5
9	Déploiement	5
9.1	Backend — Railway	5
9.2	Frontend — Render	6
10	Sécurité	6

1 Introduction

SportPulse est une application web permettant aux utilisateurs de consulter des matchs de football, publier des commentaires et effectuer des pronostics.

Le projet repose sur une architecture moderne séparant :

- un frontend React SPA ;
- un backend REST développé en Go ;
- une base SQLite ;
- une API externe football-data.org.

2 Architecture générale

Architecture globale

Frontend React SPA → API REST Go → SQLite + API football-data.org

bleuClair !80

Frontend	React + Vite	Interface utilisateur SPA
Backend	Go net/http	API REST sécurisée JWT
SQLite	mattn/go-sqlite3	Stockage des utilisateurs et pronostics
API externe	football-data.org	Données sportives temps réel

3 Choix des technologies client

Le frontend de SportPulse est développé sous forme de Single Page Application (SPA).

Technologie	Version	Rôle
React	18	Gestion de l'interface utilisateur
React Router	6	Navigation côté client
Vite	5	Bundler et serveur de développement
Tailwind CSS	4	Système de design responsive
Lucide React	latest	Icônes SVG
Fetch API	natif	Communication AJAX avec le backend

React décompose l'interface en composants réutilisables comme :

- `NavBar`
- `MatchCard`
- `ScoreInput`
- `CommentSection`

L'authentification globale est gérée via un **AuthContext** partagé à toute l'application.

4 Plan du site et navigation

Écran	Route	Accès	Description
Accueil	/	Public	Liste des matchs et statistiques
Détail match	/match/:id	Public	Scores, commentaires et pronostics
Classement	/leaderboard	Public	Classement des utilisateurs
Profil public	/profile/:id	Public	Historique d'un utilisateur
Mon profil	/me	Privé	Dashboard personnel
Inscription	/register	Public	Création de compte
Connexion	/login	Public	Authentification JWT
404	/*	Public	Page inexistante

5 Gestion des états React

Les données sont gérées principalement via :

- `useState`

- `useEffect`
- `useContext`

5.1 Chargement d'une page

1. Montage du composant
2. Déclenchement de `useEffect`
3. Appel AJAX avec `apiFetch()`
4. Réception du JSON
5. Mise à jour du state React
6. Re-render automatique

6 Authentification JWT

Sécurité

Les routes privées sont protégées par un token JWT stocké dans le `localStorage`.

6.1 Connexion

- Formulaire login
- Appel `POST /api/auth/login`
- Réception du token JWT
- Sauvegarde `localStorage`
- Redirection utilisateur

6.2 Protection des routes

La route `/me` utilise un composant `PrivateRoute` redirigeant automatiquement vers `/login` si aucun token n'est présent.

7 Gestion des événements utilisateur

7.1 Publication d'un pronostic

- Saisie des scores
- Validation du formulaire
- Appel `POST /api/matches/:id/predictions`
- Confirmation utilisateur

7.2 Publication d'un commentaire

- Saisie du commentaire
- Envoi avec touche Entrée ou bouton Send
- Mise à jour dynamique du DOM

7.3 Suppression de compte

- Ouverture modal de confirmation
- Validation utilisateur
- Appel DELETE /api/users/me
- Déconnexion automatique

8 Liste des appels AJAX

Tous les appels passent par `apiFetch(path, options)`.

Méthode	Route	Page	Auth	Corps
POST	/api/auth/register	RegisterPage	Non	{username,email,password}
POST	/api/auth/login	LoginPage	Non	{email,password}
GET	/api/matches	HomePage	Non	—
GET	/api/matches/:id	MatchDetailPage	Non	—
POST	/api/matches/:id /predictions	MatchDetailPage	Oui	{home_score_pred, away_score_pred}
PUT	/api/matches/:id /predictions	MatchDetailPage	Oui	{home_score_pred, away_score_pred}
POST	/api/matches/:id /comments	MatchDetailPage	Oui	{content}
GET	/api/leaderboard	LeaderboardPage	Non	—
GET	/api/users/me	MyProfilePage	Oui	—
DELETE	/api/users/me	MyProfilePage	Oui	—

9 Déploiement

9.1 Backend — Railway

Le backend Go est hébergé sur Railway.

- Déploiement automatique via GitHub
- Variable FOOTBALL_API_KEY
- Build :

```
1 cd server && CGO_ENABLED=1 go build -o app .
```

- Start :

```
1 ./server/app
```

URL : <https://serveurback-production.up.railway.app/api>

9.2 Frontend — Render

Le frontend React est déployé sur Render Static Site.

- Compilation Vite
- Dossier publié : dist
- Variable VITE_API_URL

```
1 npm install && npm run build
```

URL : <https://pc3r-7p1m.onrender.com>

10 Sécurité

- Authentification JWT
- Routes privées protégées
- Validation des formulaires
- Gestion des erreurs serveur
- Variables d'environnement sécurisées
- API backend séparée du frontend